Appello del 20 Luglio 2018



Politecnico di Milano

Anno accademico 2017-2018

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente specifica di una classe `Labirinto`.

```
public class Labirinto {
/*OVERVIEW: Un labirinto costituito da celle, con una sola
  entrata e una sola uscita. L'uscita e' sempre raggiungibile
  dall'entrata. La struttura del labirinto non cambia dopo la
  creazione.
  Ogni cella puo' essere occupata da un numero arbitrario
  di giocatori.
  Un giocatore si puo' muovere da una cella ad un'altra
  adiacente, muovendosi in una delle 4 direzioni cardinali
  N,S,E,O.
*/

  //genera in modo casuale un labirinto di n>=2 celle (contando in e out),
  //in cui esiste almeno un cammino da in a out (celle distinte);
  //solleva l'eccezione se i parametri non soddisfano i vincoli previsti
  public Labirinto(int n, Cella in, Cella out) throws InvalidException;

  //sotto l'ipotesi che la direzione d sia una fra N,S,O,E (precondizione)
  //restituisce la cella adiacente nella direzione data oppure la
  //cella x stessa se non ci sono celle adiacenti nella direzione data
  public Cella adiacente(Cella x, char d);

  //restituisce un elenco di tutte le celle del labirinto,
  public ArrayList<Cella> celle();

  //restituisce un elenco di tutti i giocatori presenti.
  public ArrayList<Giocatore> giocatori()

  //restituisce un cammino costituito da celle adiacenti
  //a partire dall'ingresso fino all'uscita del labirinto.
  public ArrayList<Cella> soluzione();

  //pone il giocatore g nella cella di ingresso del labirinto
  //solleva l'eccezione se g e' gia' presente nel labirinto
  public void entrata(Giocatore g) throws GiocatorePresenteException;

  //restituisce true sse g si trova nella cella di uscita del labirinto
  public boolean uscita(Giocatore g)

  //restituisce la Cella in cui si trova g
  //solleva l'eccezione se g non e' presente nel labirinto
  public Cella posizione(Giocatore g) throws GiocatoreNonPresenteException;

  //sotto l'ipotesi che il giocatore sia presente nel labirinto (precondizione)
  //muove il giocatore g di una cella nella direzione indicata
  //restituisce true se la mossa e' possibile, false altrimenti (ovvero se non ci
  //sono celle adiacenti nella direzione indicata).
  //lancia l'eccezione se la direzione non e' ammissibile (non è una tra N,S,O,E)
  public boolean muovi(Giocatore g, char d)  throws DirezioneErrataException;
}
```

Le classi `Giocatore` e `Cella` non vengono specificate non essendo rilevanti per le domande che seguono.

Domanda a)

Indicare quali metodi della classe `Labirinto` sono osservatori puri. Si scriva poi in JML un invariante pubblico per la classe `Labirinto`, individuando inoltre eventuali proprieta' che non possono essere espresse con un invariante. Si dia poi la specifica JML dei metodi `Labirinto, uscita, muovi`.

Soluzione

Osservatori: `adiacente, celle, soluzione, posizione, uscita, giocatori`
L'invariante può esprimere le seguenti condizioni: la soluzione ha almeno dimensione 2, la soluzione è contenuta nel labirinto, la posizione di ogni giocatore è parte del labirinto, il labirinto non contiene celle o giocatori ripetuti, la soluzione è formata da una sequenza di celle adiacenti.

Per semplificare le formule, introduciamo un predicato che afferma che un ArrayList x non contiene due elementi con lo stesso reference.

```
noDuplicate(x)=
(\forall int i; 0<=i && i<x.size()-1;
      !(\exists int j;i<i && j<x.size(); x.get(i)==x.get(j))
```

e un altro che afferma che gli elementi dell'ArrayList non cambiano (anche l'ordine resta uguale).

```
fixed(x)=
x.size()==\old(x.size()) &&
(\forall int i; 0<=i && i<x.size();
      x.get(i)==\old(x.get(i))
```

```
//@public invariant soluzione().size()>=2 &&
//@celle().containsAll(soluzione()) &&
//@(\forall Giocatore g; giocatori().contains(g);
//@    celle().contains(posizione(g))) &&
//@noDuplicate(celle()) && noDuplicate(giocatori()) &&
//@(\forall int i; 0<=i && i<soluzione().size()-1;
//@    (\exists char d; (d== 'N' || d== 'S' || d== 'E' || d== 'O');
//@      soluzione().get(i+1)==adiacente(soluzione().get(i),d);
```

Un invariante non e' in grado di esprimere la proprieta' evolutiva che la struttura del labirinto non cambia dopo la creazione (se non per l'inserimento o lo spostamento dei giocatori nelle celle).

Definiamo quindi come abbreviazione la proprieta' che le celle non cambiano, la soluzione non cambia e le adiacenze non cambiano:

```
nochange = fixed(celle()) && fixed(soluzione()) &&
           (\forall Cella c1; celle().contains(c1) &&
              (\forall Cella c2; celle().contains(c2);
                (\forall char d; d=="N" || d=="S" || d=="E" || d=="O";
                   c1.equals(adiacente(c2,d)) <=> c1.equals(\old(adiacente(c2,d))))
```

che dovra' essere inserita come postcondizione di tutti i metodi modificatori.

```
//@ensures(n>1 && in!=null && out!=null && !(in==out) &&
//@celle().size()==n && soluzione().size()>=2 &&
//@celle().containsAll(soluzione()) && giocatori().size()==0 &&
//@soluzione().get(0)==in && soluzione().get(soluzione().size()-1)==out;
//@signals(InvalidException e) n<2 || in==null || out == null || in==out;
public Labirinto(int n, Cella in, Cella out) throws InvalidException;

//@ensures \result==(soluzione().get(soluzione().size()-1).contains(g));
public boolean /*@ pure @*/ uscita(Giocatore g)
```

```
//@requires giocatori().contains(g);
//@ensures (d=='N' || d=='S' || d=='E' || d=='O') &&
//@nochange && posizione(g)==\old(adiacente(posizione(g),d)) &&
//@(\forall Giocatore g1; giocatori().contains(g1) && g!=g1;
//@    posizione(g1)==\old(posizione(g1)));
//@signals (DirezioneErrataException e) nochange &&
//@  fixed(giocatori()) &&
//@  (\forall Giocatore g; giocatori().contains(g);
//@     posizione(g)==\old(posizione(g))) &&
//@  (d!='N' && d!='S' && d!='E' && d!='O');
public boolean muovi(Giocatore g, char d)  throws DirezioneErrataException;
```

Domanda b)

Si consideri una variante della classe in cui il movimento può avvenire in 8 direzioni (ossia anche diagonalmente: NO, NE, SE, SO). Che cosa dovrebbe cambiare nella classe `Labirinto` (senza aggiungere nuovi metodi)?

Sarebbe possibile definire una siffatta classe come estensione di `Labirinto`? Oppure definire `Labirinto` come un'estensione di questa? Giustificare accuratamente la risposta.

Soluzione

Occorre modificare la specifica dei metodi `adiacente` e `muovi` per consentire le nuove direzioni.

Nel metodo `adiacente`, il parametro d ha una precondizione che richiede che sia una fra N,S,O,E: aggiungere le altre direzioni indebolisce la precondizione ed è quindi accettabile secondo la regola dei metodi.

Per il metodo `muovi`, invece, la postcondizione prevede il vincolo N,S,O,E: il rilassamento del vincolo la indebolisce, violando la regola dei metodi.

Quindi la nuova classe non puo' ereditare dalla precedente, a meno che non si definisca un secondo metodo per il movimento nelle altre 4 direzioni (p.es. `muoviInDiagonale`).

Anche il viceversa non e' possibile (in quanto stavolta la precondizione di `adiacente` sarebbe rafforzata).

Esercizio 2

Si consideri la seguente classe Java che realizza il tipo `Coppia`.

```
   public class Coppia {
1.    private double elem1, elem2;
2.    private Object lock;

3.    public Coppia() { elem1 = elem2 = 0; }

4.    public synchronized double get1() {
5.        return elem1;
6.    }

7.    public synchronized void set1(double val) {
8.        elem1 = val;
9.    }

10.   public void set2(double val) {
11.       synchronized(lock) {
12.           elem2 = val;
13.           notifyAll();
14.       }
15.   }
```

```
16.   public void waitForNegative() throws InterruptedException {
17.       synchronized(lock) {
18.           while(elem2>=0) wait();
19.       }
20.   }
   }
```

Domanda a)

Si indichi se le seguenti affermazioni sono vere o false, motivando brevemente la propria risposta:

1. I metodi `get1` e `set1` non possono eseguire in parallelo.

2. I metodi `set2` e `waitForNegative` non possono eseguire in parallelo.

3. I metodi `set1` e `set2` non possono eseguire in parallelo.

4. Il codice compila ed esegue senza errori. In caso negativo evidenziare. sfruttando i numeri di linea indicati nel testo, quali linee vadano modificate (e come) per evitare gli errori.

Soluzione

1. Vero, i due metodi sono sincronizzati sullo stesso oggetto (`this`).
2. Vero, i due metodi sono sincronizzati sullo stesso oggetto (`this.lock`).
3. Falso, i due metodi sono sincronizzati su oggetti diversi (`this` e `this.lock`).
4. Il codice compila correttamente ma ha un errore run-time eseguendo i metodi `set2` e `waitForNegative`. L'errore deriva dall'invocare `wait` e `notifyAll` su `this` quando si ha il lock sull'oggetto `this.lock` ma non su `this`. La soluzione prevede di sostituire la linea 13 come segue:
   `13.  lock.notifyAll()`
   Analogamente la linea 18 diventa:
   `18.  while(elem2>=0) lock.wait();`

Domanda b)

Si aggiunga, alla classe sviluppata al punto precedente, un metodo `public void set12()` che imposta il valore dell'attributo `elem2` al doppio del valore dell'attributo `elem1`. Nel fare ciò, se necessario, si modifichi il codice di sincronizzazione in maniera da evitare conflitti nell'accesso agli attributi condivisi da parte dei vari metodi (incluso il nuovo).

Soluzione

Il nuovo metodo `set12` deve accedere a entrambi gli attributi `elem1` ed `elem2`, occorre quindi sincronizzarlo rispetto a tutti gli altri metodi. Ciò può essere ottenuto cambiando il codice di sincronizzazione in maniera da sincronizzare tutti i metodi su `this` come segue:

```
public class Coppia {
    ....
    public synchronized double get1() {
        return elem1;
    }

    public synchronized void set1(double val) {
        elem1 = val;
    }

    public synchronized void set2(double val) {
        elem2 = val;
        notifyAll();
    }

    public synchronized void waitForNegative() throws InterruptedException {
        while(elem2>=0) wait();
    }

    public synchronized void set12() {
        elem2 = 2*elem1;
        notifyAll();
    }
}
```

In alternativa di può tenere la soluzione originale, con sincronizzazione separata, avendo l'accortezza, nel nuovo metodo `set12`, di acquisire il lock tanto su `this` quanto su `this.lock`, come segue:

```
public synchronized void set12() {
    synchronized(lock) {
        elem2 = 2*elem1;
        lock.notifyAll();
    }
}
```

Esercizio 3

Il diagramma UML in Figura 1 modella un calcolatore moderno e le parti di cui è tradizionalmente composto. Un calcolatore include un processore, una memoria, ed eventualmente card addizionali, ad esempio, una o più sound card. Un processore può essere una CPU oppure una accelerator card che aggiunge un determinato comportamento ad una CPU. Un processore deve essere inizializzato e può leggere/scrivere dalla memoria per caricare il programma ed eseguirlo. Una memoria può essere una RAM o una cache associata ad una RAM. Un calcolatore può gestire al massimo tre card addizionali. Per gestire una card addizionale, il calcolatore deve avere una extension board. Esistono diverse sound card con diverse funzioni.

Domanda a)

Indicare se e quali pattern UML sono stati utilizzati nel diagramma. Per ciascun pattern eventualmente individuato, indicare quali classi del diagramma sono coinvolte, quale è il ruolo del pattern, e secondo voi per quale motivo è stato applicato.

Soluzione

Si possono trovare almeno tre pattern applicati nel diagramma:

1. Proxy pattern: nel modellare l'accesso del processore alla memoria. Il proxy viene utilizzato per mascherare la differenza tra accesso in RAM e in cache.

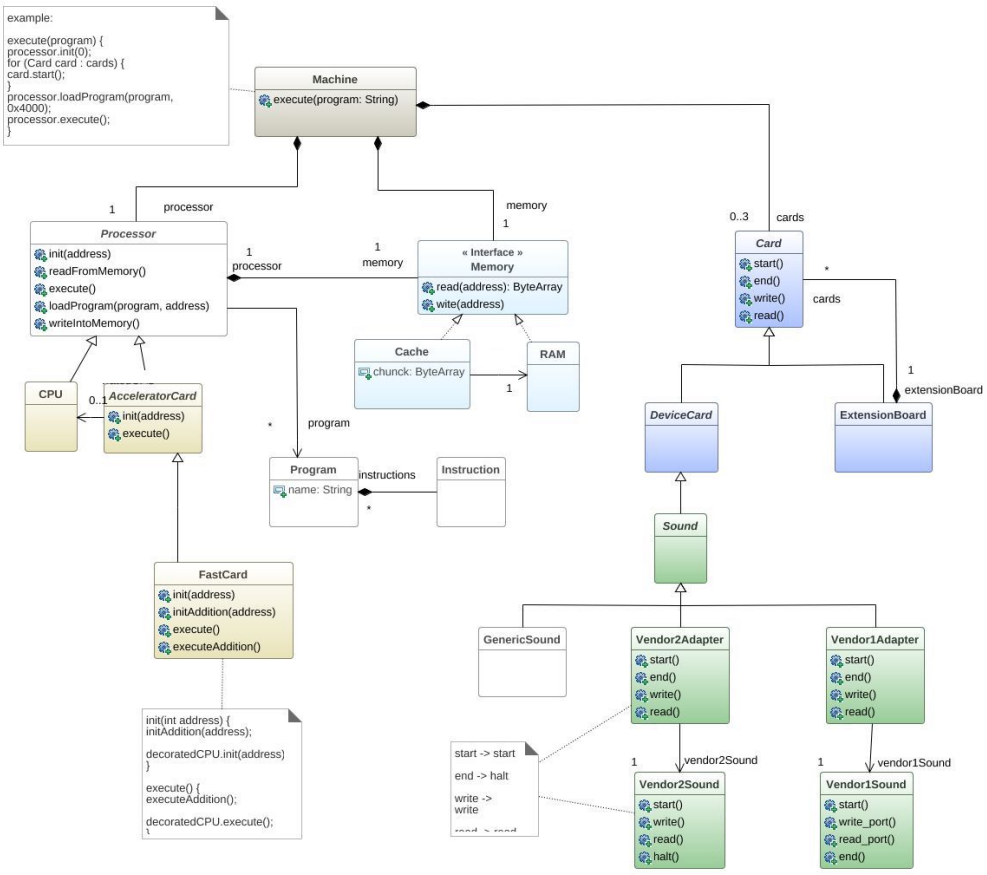


Figura 1: Diagramma UML di un moderno calcolatore.

2. Adapter pattern: nel modellare l'uso di sound card di diversi produttori. L'adapter converte comandi di alto livello nei comandi di basso livello per accedere alla sound card, che possono essere diversi da produttore a produttore.

3. Decorator pattern: nel modellare l'uso di accelerator card. Il decorator viene usato per aggiungere i comportamenti della accelerator card alla CPU esistente.

Domanda b) Estendere o modificare il diagramma in Figura 1 per modellare calcolatori con diverse *application-specific CPUs* invece che una sola CPU general purpose, ed uno *scheduler* che decide dinamicamente quale CPU deve eseguire l'istruzione corrente. Una application-specific CPU è una particolare CPU ottimizzata per eseguire solo un sottoinsieme delle istruzioni che possono comparire in un programma. Il resto del comportamento del calcolare resta quello descritto sopra. Indicare le modifiche al diagramma e se e quali pattern possono essere applicati in questo caso.

Soluzione

Si può utilizzare il pattern Strategy. Ciascuna application-specific CPU rappresenta una strategia ottimizzata per eseguire una determinata istruzione. Il comportamento dello scheduler può essere incorporato nella classe che fornisce l'interfaccia per il pattern strategy, o inserito come un componente separato ed associato allo Strategy.

Esercizio 4

Si consideri il seguente codice Java per la classe `Item`, ed una test suite composta da due test, `testOne` e `testTwo`, descritti sotto.

```
public class Item {
    private String item;
    private int size;
    public void setItem(String text)
    {
        item = text;
        size = item.length();
    }
    public void reduce()
    {
        if (size > 0)
        {
            item = item.substring(1,size);
            size = size - 1;
        }
    }
    public int getSize() { return size; }
    public String getItem() { return item; }
}
```

```
public void testOne() {
        Item item1 = new Item();
        item1.setItem("grip");
        assertEquals(4, item1.getSize());
        assertEquals("grip", item1.getItem());
        item1.reduce();
        assertEquals(3, item1.getSize());
        assertEquals("rip", item1.getItem());
    }

public void testTwo() {
        Item item1 = new Item();
        item1.setItem("");
        assertEquals(0, item1.getSize());
        assertEquals("", item1.getItem());
        item1.reduce();
        assertEquals(0, item1.getSize());
        assertEquals("", item1.getItem());
    }
```

Si risponda alle seguenti domande (a) e (b).

Domanda a)

Si calcoli la copertura della suite secondo il criterio delle istruzioni, cammini, e diramazioni (branch). Motivare la risposta.

Soluzione

La test suite che viene proposta raggiunge copertura totale secondo tutti i criteri proposti.

Domanda b)

Esiste qualche potenziale problema nel codice della classe `Item` che la suite non rileva? Quale test andrebbe aggiunto per rilevare il problema? Come andrebbe cambiata l'implementazione di `Item` per risolvere il problema?

Soluzione

Il fatto che la stringa venga inserita con un metodo `set`, affidandosi al costruttore di default per l'inizializzazione degli attributi,rende l'utilizzo della classe vulnerabile ad un eventuale `NullPointerException`, che verrebbe rivelata da un test come:

```
public void testThree() {
        Item item1 = new Item();
        assertEquals(0, item1.getSize());
        assertEquals("", item1.getItem());
    }
```

Il fix è banale, e richiede di modificare la classe aggiungendo un costruttore che prenda la stringa di test come parametro.